第5章 盛放记录的大盒子——InnoDB数据页结构

5.1 不同类型的页简介

前面章节简单介绍了页的概念。页是 InnoDB 管理存储空间的基本单位，一个页的大小一般是 16KB。InnoDB 为了不同的目的而设计了多种不同类型的页，比如存放表空间头部信息的页、存放 Change Buffer 信息的页、存放 INODE 信息的页、存放 undo 日志信息的页；等等等等。当然，如果我说的这些名词你一个都没有听过，也没有关系。我们今天也不准备说这些类型的页，我们关心的是那些存放表中记录的那种类型的页，官方称这种存放记录的页为索引（INDEX）页。鉴于我们还没有介绍过索引是什么，而这些表中的记录就是我们日常所称的数据，所以目前还是将这种存放记录的页称为数据页吧。

小贴士

别的数据库图书可能不把存放记录的页面称之为数据页，这里是为了表述方便才这么说的，大家在阅读其他图书的时候需要注意一下。

5.2 数据页结构快览

数据页代表的这块 16KB 大小的存储空间可以划分为多个部分，不同部分有不同的功能，如图 5-1 所示。

图 5-1 InnoDB 数据页结构示意图

从图 5-1 可以看出，一个 InnoDB 数据页的存储空间大致被划分成 7 个部分，有的部分占

用的字节数是确定的，有的部分占用的字节数是不确定的。下面我们通过表 5-1 大致描述一下这 7 个部分都存储一些什么内容（快速地瞅一眼就行了，后面会详细唠叨的）。

表 5-1 InnoDB 数据页结构

名称	中文名	占用空间大小	简单描述
File Header	文件头部	38 字节	页的一些通用信息
Page Header	页面头部	56 字节	数据页专有的一些信息
Infimum + Supremum	页面中的最小记录和最大记录	26 字节	两个虚拟的记录
User Records	用户记录	不确定	用户存储的记录内容
Free Space	空闲空间	不确定	页中尚未使用的空间
Page Directory	页目录	不确定	页中某些记录的相对位置
File Trailer	文件尾部	8 字节	校验页是否完整

小贴士

我们接下来并不打算按照页中各个部分的出现顺序来依次介绍它们，因为按照顺序介绍的话会把大量陌生的概念一股脑儿地呈现在大家面前，严重打击各位的信心与兴趣。现在还是希望大家的学习曲线能够平缓一点儿，由浅入深一点儿，希望大家能接受这种写作手法。

5.3 记录在页中的存储

在页的 7 个组成部分中，我们自己存储的记录会按照指定的行格式存储到 User Records 部分。但是在一开始生成页的时候，其实并没有 User Records 部分，每当插入一条记录时，都会从 Free Space 部分（也就是尚未使用的存储空间）申请一个记录大小的空间，并将这个空间划分到 User Records 部分。当 Free Space 部分的空间全部被 User Records 部分替代掉之后，也就意味着这个页使用完了，此时如果还有新的记录插入，就需要去申请新的页了。这个过程如图 5-2 所示。

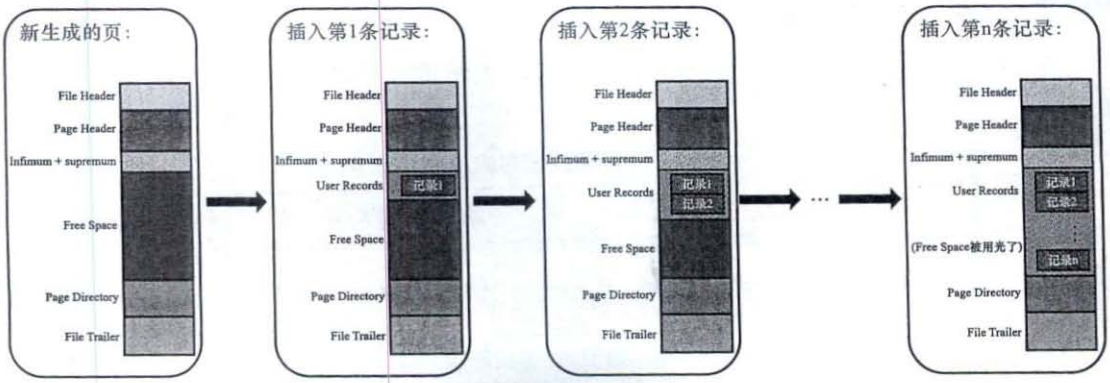

图 5-2 记录在页中的存储

为了更好地管理 User Records 中的这些记录，设计 InnoDB 的大叔可费了一番力气。他们将力气费在哪里了呢？不就是把记录按照指定的行格式一条一条摆在 User Records 部分么？其实这么说也没啥问题，但是“魔鬼藏在细节中”，我们还得从记录行格式的记录头信息说起。

5.3.1　记录头信息的秘密

为了故事的顺利发展，我们先创建一个表：

```
mysql> CREATE TABLE page_demo(
    ->     c1 INT,
    ->     c2 INT,
    ->     c3 VARCHAR(10000),
    ->     PRIMARY KEY (c1)
    -> ) CHARSET=ascii ROW_FORMAT=COMPACT;
Query OK, 0 rows affected (0.03 sec)
```

这个新创建的 page_demo 表有 3 列，其中 c1 和 c2 列用来存储整数，c3 列用来存储字符串。需要注意的是，我们把 c1 列指定为主键，所以 InnoDB 就没必要创建那个所谓的 row_id 隐藏列了。而且我们为这个表指定了 ascii 字符集以及 COMPACT 的行格式，所以这个表中记录的行格式示意图如图 5-3 所示。

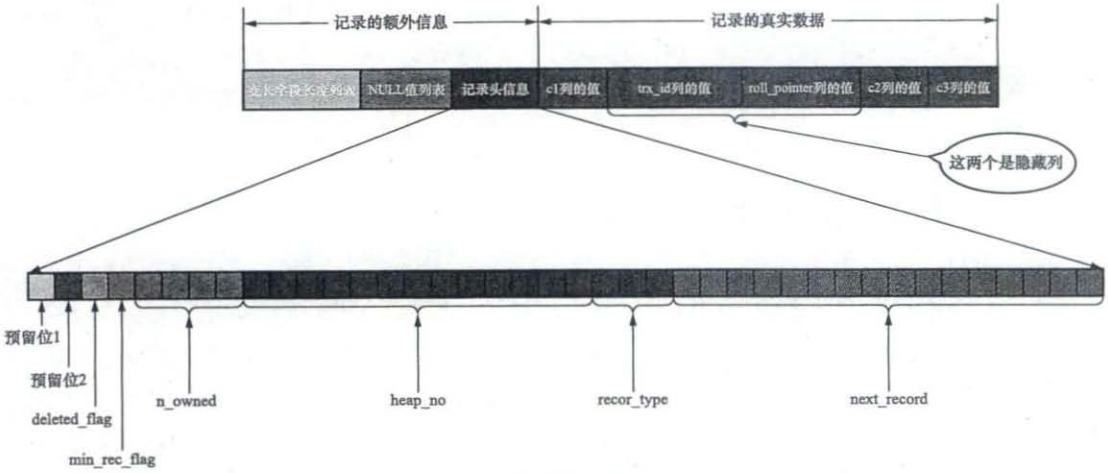

图 5-3　COMPACT 行格式示意图

从图 5-3 可以看到，我们特意把记录头信息的 5 字节的数据给标出来了，说明它很重要。我们再次浏览一下这些记录头信息中各个属性的大体意思（目前使用 COMPACT 行格式进行演示），如表 5-2 所示。

表 5-2　记录头信息的属性及描述

名称	大小（比特）	描述
预留位 1	1	没有使用
预留位 2	1	没有使用
deleted_flag	1	标记该记录是否被删除
min_rec_flag	1	B+ 树中每层非叶子节点中的最小的目录项记录都会添加该标记
n_owned	4	一个页面中的记录会被分成若干个组，每个组中有一个记录是“带头大哥”，其余的记录都是“小弟”。“带头大哥”记录的 n_owned 值代表该组中所有的记录条数，“小弟”记录的 n_owned 值都为 0

续表

名称	大小（比特）	描述
heap_no	13	表示当前记录在页面堆中的相对位置
record_type	3	表示当前记录的类型，0 表示普通记录，1 表示 B+ 树非叶节点的目录项记录，2 表示 Infimum 记录，3 表示 Supremum 记录
next_record	16	表示下一条记录的相对位置

由于我们现在主要在唠叨记录头信息的作用，所以为了方便大家理解，我们只在 page_demo 表的行格式演示图中（见图 5-4）画出有关的头信息属性以及 c1、c2、c3 列的信息（其他信息没画不代表它们不存在，只是为了理解上的方便而在图中省略了）。

图 5-4 page_demo 表的行格式简化图

下面试着向 page_demo 表中插入几条记录：

```
mysql> INSERT INTO page_demo VALUES(1, 100, 'aaaa'), (2, 200, 'bbbb'), (3, 300, 'cccc'),
(4, 400, 'dddd');
Query OK, 4 rows affected (0.00 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

为了方便大家分析这些记录在页的 User Records 部分是怎么表示的，这里把记录中的头信息和实际的列数据都用十进制表示出来了（其实是一堆二进制位）。这些记录的示意图如图 5-5 所示。

在查看图 5-5 时需要注意一下，各条记录在 User Records 中存储的时候并没有空隙，这里只是为了方便大家观看才把每条记录单独画在一行中。我们现在对照着图 5-5 来看看记录头信息中的各个属性是什么意思。

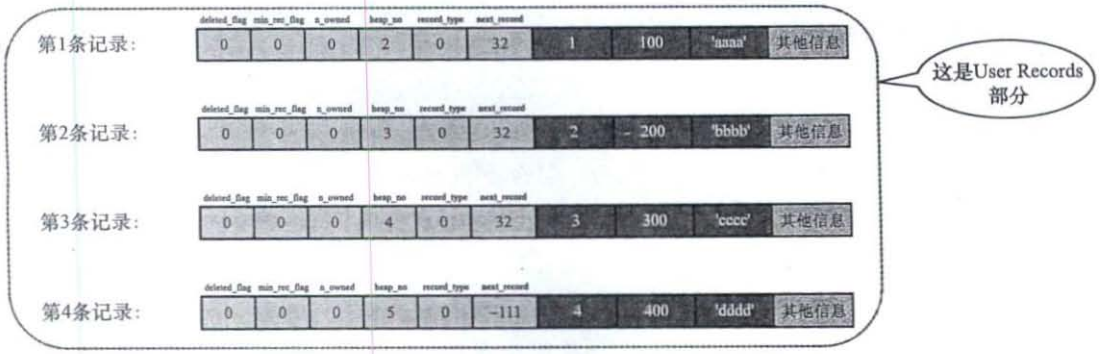

图 5-5 记录在页的 User Records 部分的存储结构

- deleted_flag：这个属性用来标记当前记录是否被删除，占用 1 比特。值为 0 时表示记

录没有被删除，值为 1 时表示记录被删除了。

啥？被删除的记录还在页中么？是的，摆在台面上的和背地里做的可能大相径庭。你以为记录被删除了，可它还在真实的磁盘上。这些被删除的记录之所以不从磁盘上移除，是因为在移除它们之后，还需要在磁盘上重新排列其他的记录，这会带来性能消耗，所以只打一个删除标记就可以避免这个问题。所有被删除掉的记录会组成一个垃圾链表，记录在这个链表中占用的空间称为可重用空间（关于链表是怎么形成的，在介绍过 next_record 属性后大家就知道了）。之后若有新记录插入到表中，它们就可能覆盖掉被删除的这些记录占用的存储空间。

小贴士

将 deleted_flag 属性设置为 1 和将被删除的记录加入到垃圾链表中其实是两个阶段，后面在介绍 undo 日志时会详细唠叨删除操作的详细执行过程，现在少安毋躁。

- min_rec_flag：B+ 树每层非叶子节点中的最小的目录项记录都会添加该标记。什么是 B+ 树？什么是非叶子节点？什么是目录项记录？好吧，等第 6 章介绍索引的时候再聊这些话题。我们现在只需要知道自己插入的 4 条记录的 min_rec_flag 值都是 0，意味着它们都不是 B+ 树非叶子节点中的最小的目录项记录。
- n_owned：这个暂时保密，稍后它是主角。
- heap_no：我们向表中插入的记录从本质上来说都是放到数据页的 User Records 部分，这些记录一条一条地亲密无间地排列着，如图 5-6 所示（这不是 page_demo 表中的记录，只是一个通用的示意图）。

图 5-6　记录在数据页的 User Records 中的排放方式

设计 InnoDB 的大叔把记录一条一条亲密无间排列的结构称之为堆（heap）。为了方便管理这个堆，他们把一条记录（这条记录的 deleted_flag 可以为 1）在堆中的相对位置称之为 heap_no。在页面前边的记录 heap_no 相对较小，在页面后边的记录 heap_no 相对较大，每新申请一条记录的存储空间时，该条记录比物理位置在它前边的那条记录的 heap_no 值大 1。从图 5-5 所示的 page_demo 表的各条记录示意图中可以看出，我们插入的 4 条记录的 heap_no 属性值分别是 2、3、4、5。是不是少了点啥？是的，怎么不见 heap_no 值为 0 和 1 的记录呢？

这其实是设计 InnoDB 的大叔玩的一个小把戏，他们自动给每个页里面加了两条记录，由于这两条记录并不是用户自己插入的，所以有时候也称为伪记录或者虚拟记录。在这两条伪记录中，一条代表页面中的最小记录（也可以写作 Infimum 记录），另外一条代表页面中的最大记录（也可以写作 Supremum 记录）。这两条伪记录也算作堆的一部分（很显然这两条伪记录的 heap_no 值最小，说明它们在页面中的相对位置最靠前）。

等一下，记录可以比大小么？

是的，记录也可以比大小。对于一条完整的记录来说，比较记录的大小就是比较主键的大小。比如，我们插入的 4 条记录的主键值分别是 1、2、3、4，这也就意味着这 4 条记录从小到大依次递增。

小贴士

请注意，前文强调的是对于“一条完整的记录”来说，比较记录的大小相当于比的是主键的大小。下一章还会介绍只存储一条记录的部分列的情况，敬请期待。

但是，无论我们向页中插入了多少条记录，设计 InnoDB 的大叔都规定，任何用户记录都比 Infimum 记录大，任何用户记录都比 Supremum 记录小。

小贴士

虽然 Infimum 记录和 Supremum 记录没有主键值，但是设计 InnoDB 的大叔规定：Infimum 记录是一个页面中最小的记录，Supremum 记录是一个页面中最大的记录。这是规定！规定！规定！

Infimum 和 Supremum 这两条记录的构造十分简单，都是由 5 字节大小的记录头信息和 8 字节大小的一个固定单词组成的，如图 5-7 所示。

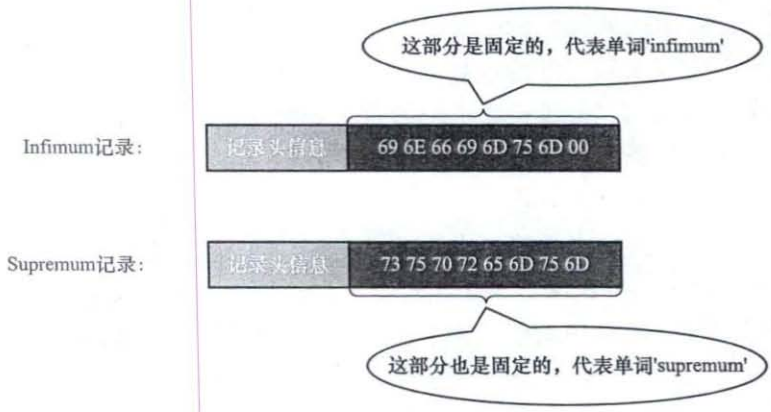

图 5-7 Infimum 和 Supremum 记录的结构

由于 Infimum 和 Supremum 这两条记录是设计 InnoDB 的大叔默认创建的记录，为了与用户自己插入的记录进行区分，我们就不把它们存放在页的 User Records 部分，而是单独放在一个称为 Infimum + Supremum 的部分，如图 5-8 所示。

从图 5-8 可以看出，Infimum 记录和 Supremum 记录的 heap_no 值分别是 0 和 1，也就是说它们在堆中的相对位置最靠前。另外还需要注意的一点是，堆中记录的 heap_no 值在分配之后就不会发生改动了，即使之后删除了堆中的某条记录，这条被删除记录的 heap_no 值也仍然保持不变。

- record_type：这个属性表示当前记录的类型。一共有 4 种类型的记录，其中 0 表示普通记录，1 表示 B+ 树非叶节点的目录项记录，2 表示 Infimum 记录，3 表示 Supremum 记录。从图 5-8 也可以看出，我们自己插入的记录就是普通记录，它们的 record_type 值都是 0，而 Infimum 记录和 Supremum 记录的 record_type 值分别为 2 和 3。至于 record_type 为 1 的情况，我们之后在唠叨索引的时候会重点介绍的。

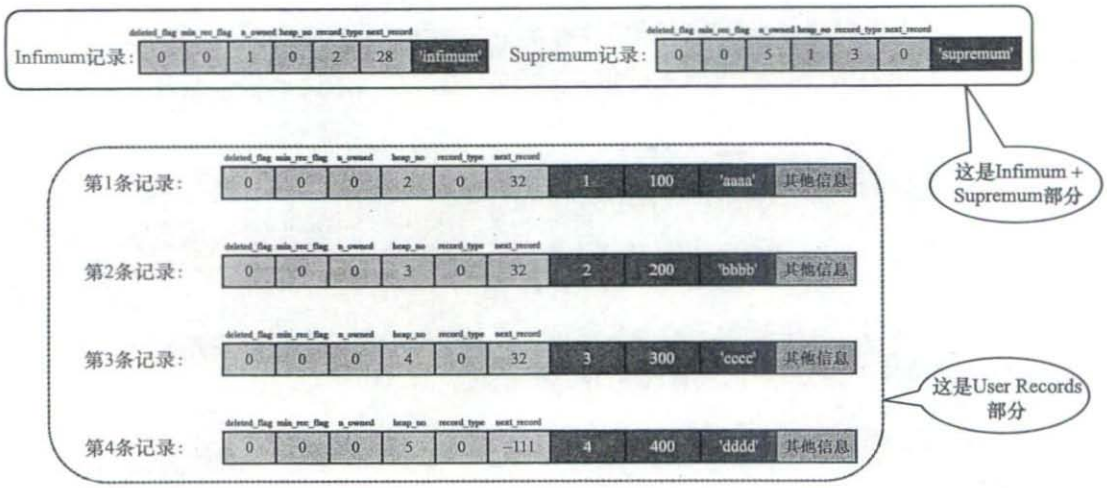

图 5-8　记录存放方式

- next_record：这个属性非常重要，它表示从当前记录的真实数据到下一条记录的真实数据的距离。如果该属性值为正数，说明当前记录的下一条记录在当前记录的后面；如果该属性值为负数，说明当前记录的下一条记录在当前记录的前面。比如，第 1 条记录的 next_record 值为 32，意味着从第 1 条记录的真实数据的地址处向后找 32 字节便是下一条记录的真实数据。再比如，第 4 条记录的 next_record 值为 −111，意味着从第 4 条记录的真实数据的地址处向前找 111 字节便是下一条记录的真实数据。如果大家熟悉数据结构的话，就会立即明白这其实就是个链表，可以通过一条记录找到它的下一条记录。但是需要注意的一点是，下一条记录指的并不是插入顺序中的下一条记录，而是按照主键值由小到大的顺序排列的下一条记录。而且规定 Infimum 记录的下一条记录就是本页中主键值最小的用户记录，本页中主键值最大的用户记录的下一条记录就是 Supremum 记录。为了更形象地表示这个 next_record 属性的作用，我们用箭头来替代 next_record 中的值（注意箭头指向的位置，每个箭头都指向记录的真实数据开始的地方），如图 5-9 所示。

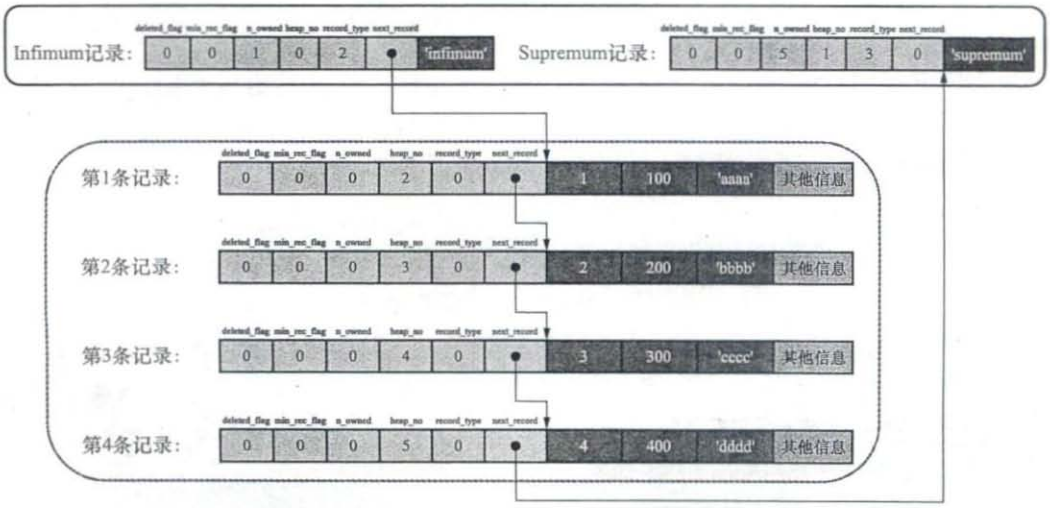

图 5-9　使用箭头替代 next_record 的值

从图 5-9 可以看出，记录按照主键从小到大的顺序形成了一个单向链表。Supremum 记录的 next_record 值为 0，也就是说 Supremum 记录之后就没有下一条记录了，这也意味着 Supremum 记录就是这个单向链表中的最后一个节点。如果从表中删除一条记录，这个由记录组成的单向链表也是会跟着变化，比如我们把第 2 条记录删掉：

```
mysql> DELETE FROM page_demo WHERE c1 = 2;
Query OK, 1 row affected (0.02 sec)
```

删掉第 2 条记录后的示意图如图 5-10 所示。

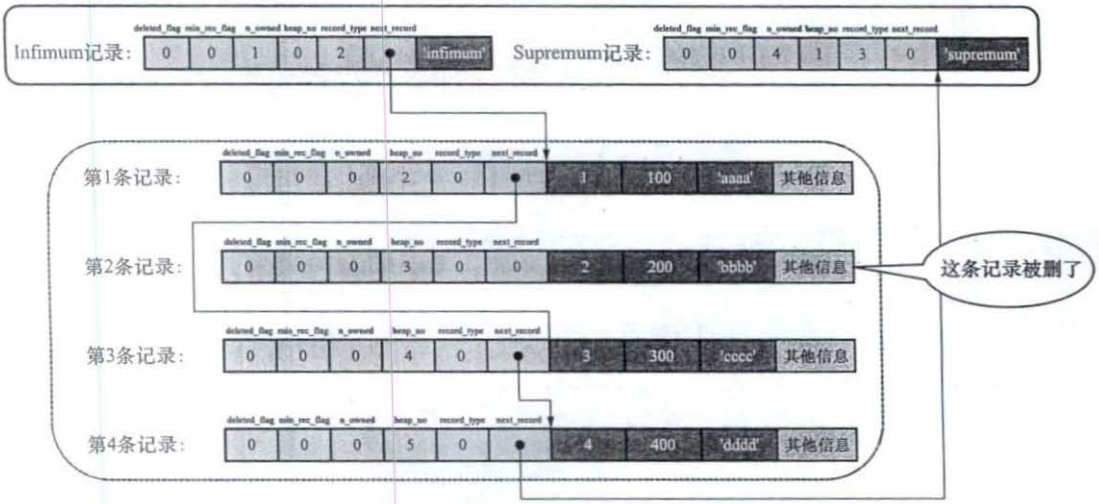

图 5-10 删掉第 2 条记录后的示意图

从图 5-10 可以看出，删除第 2 条记录后主要发生了下面这些变化：

- 第 2 条记录并没有从存储空间中移除，而是把该条记录的 deleted_flag 值设置为 1；
- 第 2 条记录的 next_record 值变为 0，意味着该记录没有下一条记录了；
- 第 1 条记录的 next_record 指向了第 3 条记录；
- 还有一点大家可能忽略了，那就是 Supremum 记录的 n_owned 值从 5 变成了 4。关于这一点变化稍后会详细介绍。

所以，无论怎么对页中的记录进行增删改操作，InnoDB 始终会维护记录的一个单向链表，链表中的各个节点是按照主键值由小到大的顺序链接起来的。

小贴士

大家会不会觉得 next_record 这个指针有点儿奇怪，它为啥要指向记录头信息和真实数据之间的位置呢？为啥不干脆指向整条记录的开头位置，也就是记录的额外信息开头的位置呢？原因是这个位置刚刚好，向左读取就是记录头信息，向右读取就是真实数据。前文还说过，变长字段长度列表、NULL 值列表中的信息都是逆序存放的，这样可以使记录中位置靠前的字段和它们对应的字段长度信息在内存中的距离更近，这可能会提高高速缓存的命中率。

再来看一个有意思的事情：主键值为 2 的记录被删掉了，但是却没有回收存储空间（该记录的 heap_no 也未发生改变），如果我们再次把这条记录插入到表中，会发生什么呢？

```
mysql> INSERT INTO page_demo VALUES(2, 200, 'bbbb');
Query OK, 1 row affected (0.00 sec)
```

我们看一下记录的存储情况，如图 5-11 所示。

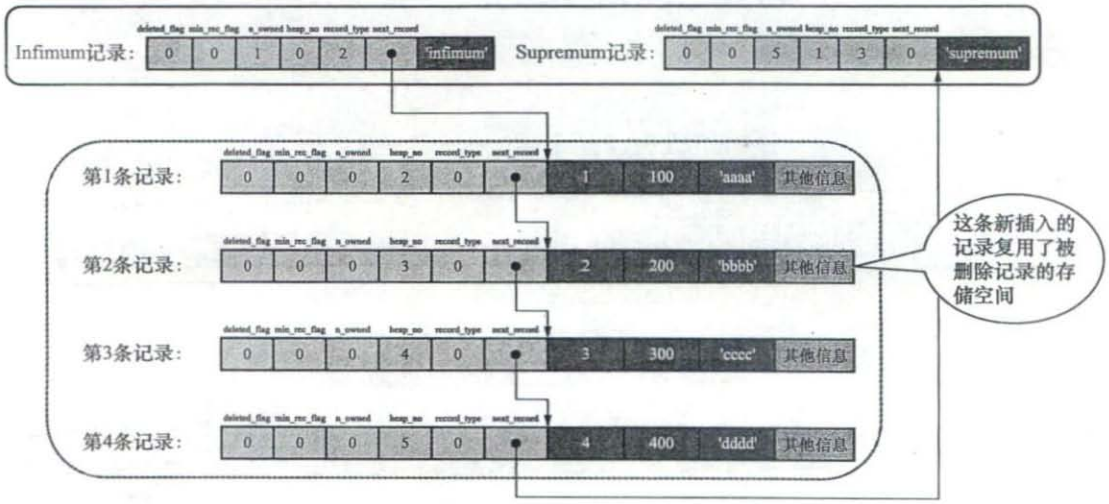

图 5-11 再次将第 2 条记录插入后，记录的存储情况

从图 5-11 可以看到，InnoDB 并没有因为新记录的插入而为它申请新的存储空间，而是直接复用了原来被删除记录的存储空间。

小贴士

当数据页中存在多条被删除的记录时，可以使用这些记录的 next_record 属性将这些被删除的记录组成一个垃圾链表，以备之后重用这部分存储空间。关于垃圾链表的更多信息，第 20 章会进行超级详细的描述，现在先蜻蜓点水般地知道这些就好了，一口吃不成个胖子。

5.4 Page Directory（页目录）

现在，我们知道了记录在页中是按照主键值由小到大的顺序串联成一个单向链表，如果想根据主键值查找页中的某条记录，该咋办呢？比如说下面这样的查询语句：

```
SELECT * FROM page_demo WHERE c1 = 3;
```

最笨的办法是从 Infimum 记录开始，沿着单向链表一直往后找，总有一天会找到（或者找不到）。而且在找的时候还能投机取巧，因为链表中各个记录的值是按照从小到大的顺序排列的，所以当链表中某个节点代表的记录的主键值大于想要查找的主键值时，就可以停止查找了（因为该节点后边的节点的主键值依次递增）。

当页中存储的记录数量比较少时，这种方法用起来也没有啥问题。比如，我们的表中只插入了 4 条自己的记录，所以最多找 4 次就可以把所有记录都遍历一遍。但是，如果一个页中存储了非常多的记录，遍历操作对性能来说还是有损耗的，所以说这种遍历查找是一个笨办法。但是设计 InnoDB 的大叔是什么人，他们能用这么笨的办法么，当然是要设计一种更好的查找

方式了。他们从图书的目录中找到了灵感。

我们平时在一本书中查找某个内容的时候，一般会先看目录，找到该内容对应的图书页码，然后再到对应的页码去查看内容。设计 InnoDB 的大叔也为我们的记录制作了一个类似的目录。制作过程如下所示。

1. 将所有正常的记录（包括 Infimum 和 Supremum 记录，但不包括已经移除到垃圾链表的记录）划分为几个组。
2. 每个组的最后一条记录（也就是组内最大的那条记录）相当于“带头大哥”，组内其余的记录相当于“小弟”。“带头大哥”记录的头信息中的 n_owned 属性表示该组内共有几条记录。
3. 将每个组中最后一条记录在页面中的地址偏移量（就是该记录的真实数据与页面中第 0 个字节之间的距离）单独提取出来，按顺序存储到靠近页尾部的地方。这个地方就是 Page Directory（页目录，见图 5-1）。页目录中的这些地址偏移量称为槽（Slot），每个槽占用 2 字节。页目录就是由多个槽组成的。

小贴士

一个正常的页面也就是 16KB 大小，即 16384 字节，而 2 字节可以表示的地址偏移量范围是 0 ～ 65535，所以用 2 字节表示一个槽足够了。

比如，现在 page_demo 表中正常的记录共有 6 条。InnoDB 会把它们分成 2 个组，第一组只有一个 Infimum 记录，第二组是剩余的 5 条记录。2 个组就对应着 2 个槽，每个槽中存放每个组中最大的那条记录在页面中的地址偏移量，如图 5-12 所示。

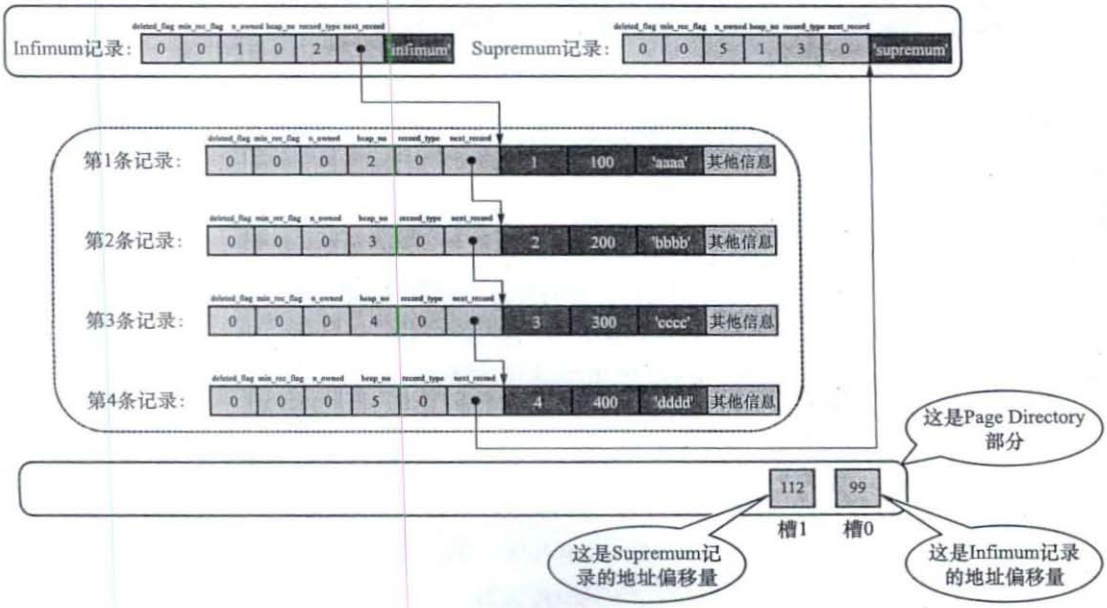

图 5-12 page_demo 表中的记录排列方式

在图 5-12 中需要注意下面几点。

- 页目录部分中有 2 个槽，也就意味着记录被分成了 2 个组。槽 1 中的值是 112，代表 Supremum 记录在页面中的地址偏移量（就是从页面的 0 字节开始数，数 112 字节）；

槽 0 中的值是 99，代表 Infimum 记录的地址偏移量。

- 注意 Infimum 记录和 Supremum 记录的头信息中的 n_owned 属性。
  - Infimum 记录的 n_owned 值为 1，这表示以 Infimum 记录为最后一个节点的这个分组中只有 1 条记录，也就是 Infimum 记录自身。
  - Supremum 记录的 n_owned 值为 5，这表示以 Supremum 记录为最后一个节点的这个分组中有 5 条记录，即除了 Supremum 记录自身之外，还有我们插入的 4 条记录。
- 每个槽占用 2 字节，按照对应记录的大小相邻分布。槽对应的记录越小，它的位置越靠近 File Trailer。

99 和 112 这样的地址偏移量很不直观，我们用箭头指向的方式替代数字，这样更容易理解。修改后的记录排列方式示意图如图 5-13 所示。

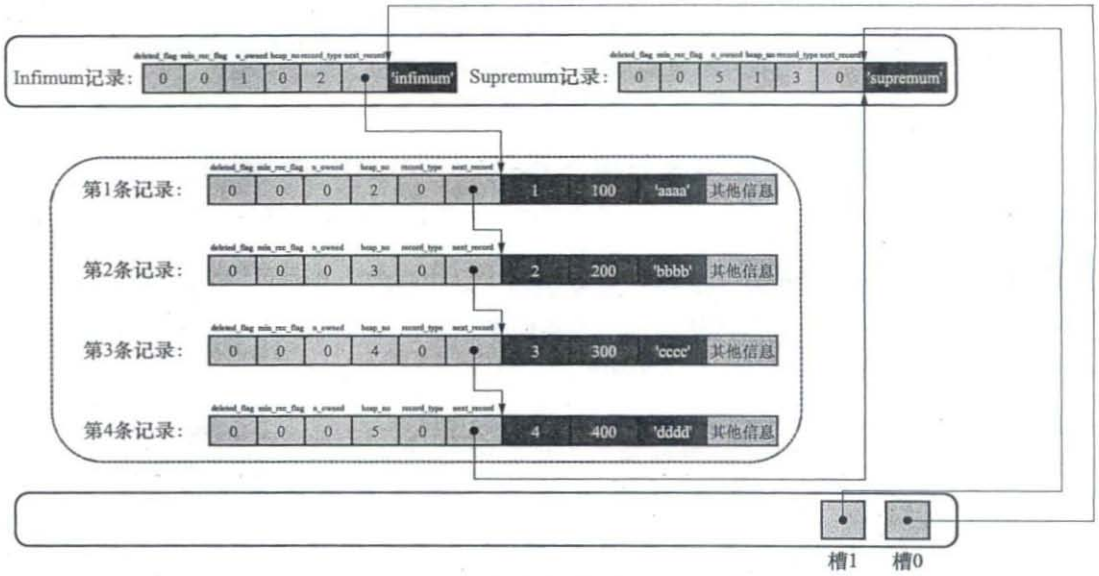

图 5-13　用箭头替代槽中数字的示意图

怎么看上去怪怪的呢？这么乱的图对于我这个强迫症患者真是不能忍，我们暂时先不管各条记录在存储设备上的排列方式了，单纯从逻辑上看一下这些记录和页目录的关系，如图 5-14 所示。

这样一来就顺眼多了！不过划分分组的依据是什么呢，也就是说，为什么 Infimum 记录的 n_owned 值为 1，而 Supremum 记录的 n_owned 值为 5 呢？这里面有什么猫腻？

是的，设计 InnoDB 的大叔对每个分组中的记录条数是有规定的：对于 Infimum 记录所在的分组只能有 1 条记录，Supremum 记录所在的分组拥有的记录条数只能在 1 ～ 8 条之间，剩下的分组中记录的条数范围只能是在 4 ～ 8 条之间。所以给记录进行分组是按照下面的步骤进行的。

1. 在初始情况下，一个数据页中只有 Infimum 记录和 Supremum 记录这两条，它们分属于两个分组。页目录中也只有两个槽，分别代表 Infimum 记录和 Supremum 记录在页面中的地址偏移量。
2. 之后每插入一条记录，都会从页目录中找到对应记录的主键值比待插入记录的主键值大并且差值最小的槽（从本质上来说，槽是一个组内最大的那条记录在页面中的地址偏移量，通过槽可以快速找到对应的记录的主键值），然后把该槽对应的记录的 n_owned 值

加 1，表示本组内又添加了一条记录，直到该组中的记录数等于 8 个。

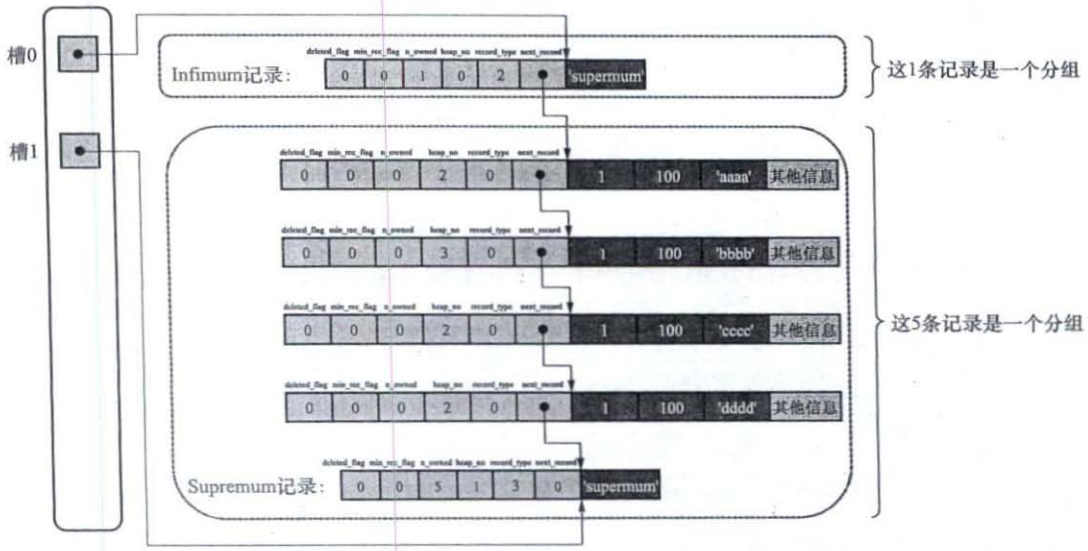

图 5-14 记录和页目录的关系

小贴士

再次强调，对于 Infimum 记录和 Supremum 记录来说，它们虽然没有主键值，但是人为规定它们是一个页面中最小和最大的记录。

3. 当一个组中的记录数等于 8 后，再插入一条记录，会将组中的记录拆分成两个组，其中一个组中 4 条记录，另一个 5 条记录。这个拆分过程会在页目录中新增一个槽，记录这个新增分组中最大的那条记录的偏移量。

由于现在 page_demo 表中的记录太少，无法演示在添加页目录之后是如何加快查找速度的，所以我们再往 page_demo 表中添加一些记录：

```
mysql> INSERT INTO page_demo VALUES(5, 500, 'eeee'), (6, 600, 'ffff'), (7, 700, 'gggg'),
(8, 800, 'hhhh'), (9, 900, 'iiii'), (10, 1000, 'jjjj'), (11, 1100, 'kkkk'), (12, 1200, 'llll'),
(13, 1300, 'mmmm'), (14, 1400, 'nnnn'), (15, 1500, 'oooo'), (16, 1600, 'pppp');
Query OK, 12 rows affected (0.00 sec)
Records: 12  Duplicates: 0  Warnings: 0
```

我们一口气往表中添加了 12 条记录，现在页中就一共有 18 条记录了（包括 Infimum 和 Supremum 记录）。这些记录被分成了 5 个组，如图 5-15 所示。

小贴士

我们这里在插入记录时是按照主键值从小到大的顺序依次插入的。如果插入的记录的主键值没有顺序，你可以试试按照我们刚才唠叨过的给记录进行分组的步骤推演一下，页目录将会变成什么样子。

因为把 16 条记录的全部信息都画在一张图中太占地方，也容易让人眼花，所以这里也省略了各个记录之间的箭头（图里没画不代表没有！），只保留了用户记录头信息中的 n_owned 和 next_record 属性。现在看看怎么在这个页目录中查找记录。因为一个槽占用 2 字节，各个

槽之间是挨着的，而且它们代表的记录的主键值都是从小到大排序的，所以可以使用二分法快速查找。5 个槽的编号分别是 0、1、2、3、4，所以初始情况下最低的槽就是 low=0，最高的槽就是 high=4。比如，我们想找主键值为 6 的记录，过程是这样的。

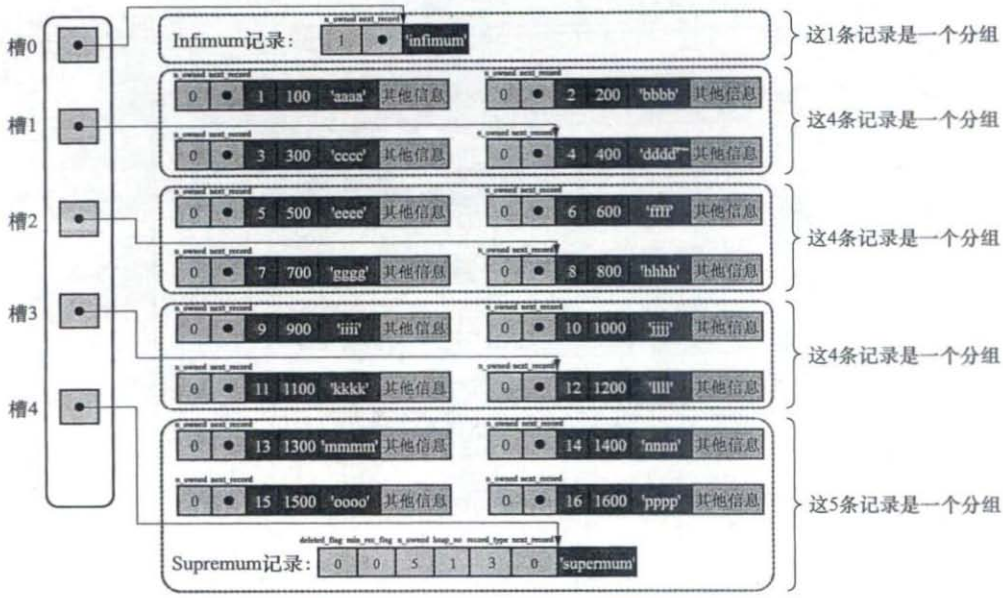

图 5-15　向 page_demo 表中添加记录

1. 计算中间槽的位置：(0+4)/2=2，查看槽 2 对应记录的主键值为 8；又因为 8 > 6，所以设置 high=2，low 保持不变。
2. 重新计算中间槽的位置：(0+2)/2=1，查看槽 1 对应记录的主键值为 4；又因为 4 < 6，所以设置 low=1，high 保持不变。
3. 因为 high − low 的值为 1，所以确定主键值为 6 的记录在槽 2 对应的组中。此时需要找到槽 2 所在分组中主键值最小的那条记录，然后沿着单向链表遍历槽 2 中的记录。但是前文又说过，每个槽对应的记录都是该组中主键值最大的记录，这里槽 2 对应的记录是主键值为 8 的记录，怎么定位一个组中最小的记录呢？别忘了各个槽都是挨着的，我们可以很轻易地找到槽 1 对应的记录（主键值为 4），这条记录的下一条记录就是槽 2 所在分组中主键值最小的记录，其主键值为 5。所以，我们可以从这条主键值为 5 的记录出发，遍历槽 2 中的各条记录，直到找到主键值为 6 的那条记录即可。由于一个组中包含的记录条数最多是 8 条，所以遍历一个组中的记录的代价是很小的。

综上所述，在一个数据页中查找指定主键值的记录时，过程分为两步。

1. 通过二分法确定该记录所在分组对应的槽，然后找到该槽所在分组中主键值最小的那条记录。
2. 通过记录的 next_record 属性遍历该槽所在的组中的各个记录。

小贴士

如果你不知道二分法是什么，找本基础算法书看看吧。

5.5 Page Header（页面头部）

设计 InnoDB 的大叔为了能得到存储在数据页中的记录的状态信息，比如数据页中已经存储了多少条记录、Free Space 在页面中的地址偏移量、页目录中存储了多少个槽等，特意在数据页中定义了一个名为 Page Header 的部分，它是页结构的第 2 部分（见图 5-1），占用固定的 56 字节，专门存储各种状态信息。Page Header 中各个字节的具体用途如表 5-3 所示。

表 5-3 Page Header 的结构及描述

状态名称	占用空间大小	描述
PAGE_N_DIR_SLOTS	2 字节	在页目录中的槽数量
PAGE_HEAP_TOP	2 字节	还未使用的空间最小地址，也就是说从该地址之后就是 Free Space
PAGE_N_HEAP	2 字节	第 1 位表示本记录是否为紧凑型的记录，剩余的 15 位表示本页的堆中记录的数量（包括 Infimum 和 Supremum 记录以及标记为“已删除”的记录）
PAGE_FREE	2 字节	各个已删除的记录通过 next_record 组成一个单向链表，这个单向链表中的记录所占用的存储空间可以被重新利用；PAGE_FREE 表示该链表头节点对应记录在页面中的偏移量
PAGE_GARBAGE	2 字节	已删除记录占用的字节数
PAGE_LAST_INSERT	2 字节	最后插入记录的位置
PAGE_DIRECTION	2 字节	记录插入的方向
PAGE_N_DIRECTION	2 字节	一个方向连续插入的记录数量
PAGE_N_RECS	2 字节	该页中用户记录的数量（不包括 Infimum 和 Supremum 记录以及被删除的记录）
PAGE_MAX_TRX_ID	8 字节	修改当前页的最大事务 id，该值仅在二级索引页面中定义
PAGE_LEVEL	2 字节	当前页在 B+ 树中所处的层级
PAGE_INDEX_ID	8 字节	索引 ID，表示当前页属于哪个索引
PAGE_BTR_SEG_LEAF	10 字节	B+ 树叶子节点段的头部信息，仅在 B+ 树的根页面中定义
PAGE_BTR_SEG_TOP	10 字节	B+ 树非叶子节点段的头部信息，仅在 B+ 树的根页面中定义

如果大家认真看过前文，一定会清楚从 PAGE_N_DIR_SLOTS 到 PAGE_LAST_INSERT 以及 PAGE_N_RECS 的意思。如果不清楚，你应该回头再看一遍前文。剩下的状态信息看不明白也不要着急，饭要一口一口吃，东西要一点一点学（一定要沉住气，不要被这些名词吓到）。

在这里我们先唠叨 PAGE_DIRECTION 和 PAGE_N_DIRECTION 的意思。

- PAGE_DIRECTION：假如新插入的一条记录的主键值比上一条记录的主键值大，我们说这条记录的插入方向是右边，反之则是左边。用来表示最后一条记录插入方向的状态就是 PAGE_DIRECTION。
- PAGE_N_DIRECTION：假设连续几次插入新记录的方向都是一致的，InnoDB 会把沿着同一个方向插入记录的条数记下来，这个条数就用 PAGE_N_DIRECTION 状态表示。当然，如果最后一条记录的插入方向发生了改变，这个状态的值会被清零后重新统计。

至于还没提到的那些状态，现在大家还不需要知道。不要着急，当我们学完了后面的内容后再回头来看，就会发现一切都是那么清晰。

5.6 File Header（文件头部）

前文唠叨的 Page Header 专门针对的是数据页记录的各种状态信息，比如页里有多少条记录，有多少个槽。现在介绍的 File Header 通用于各种类型的页，也就是说各种类型的页都会以 File Header 作为第一个组成部分，它描述了一些通用于各种页的信息，比如这个页的编号是多少，它的上一个页和下一个页是谁；等等等等。File Header 部分占用固定的 38 字节，由表 5-4 所示的内容组成。

表 5-4 File Header 的结构及描述

状态名称	占用空间大小	描述
FIL_PAGE_SPACE_OR_CHKSUM	4 字节	当 MySQL 的版本低于 4.0.14 时，该属性表示本页面所在的表空间 ID；在之后的版本中，该属性表示页的校验和（checksum）
FIL_PAGE_OFFSET	4 字节	页号
FIL_PAGE_PREV	4 字节	上一个页的页号
FIL_PAGE_NEXT	4 字节	下一个页的页号
FIL_PAGE_LSN	8 字节	页面被最后修改时对应的 LSN（Log Sequence Number，日志序列号）值
FIL_PAGE_TYPE	2 字节	该页的类型
FIL_PAGE_FILE_FLUSH_LSN	8 字节	仅在系统表空间的第一个页中定义，代表文件至少被刷新到了对应的 LSN 值
FIL_PAGE_ARCH_LOG_NO_OR_SPACE_ID	4 字节	页属于哪个表空间

对照着表 5-4，我们看几个目前比较重要的部分。

- FIL_PAGE_SPACE_OR_CHKSUM：MySQL 4.0.14 以下的版本应该没人用了吧，我们直接讨论高于 4.0.14 版本的情况。这个属性代表当前页面的校验和（checksum）。啥是校验和？就是对于一个很长的字节串来说，我们会通过某种算法计算出一个比较短的

值来代表这个很长的字节串，这个比较短的值就称为校验和。这样在比较两个很长的字节串之前，先比较这两个长字节串的校验和。如果校验和都不一样，则两个长字节串肯定是不同的，这样就省去了直接比较两个长字节串的时间损耗。

- FIL_PAGE_OFFSET：每一个页都有一个单独的页号，如同我们的身份证号码一样。InnoDB 通过页号来唯一定位一个页。
- FIL_PAGE_TYPE：表示当前页的类型。前文说过，InnoDB 为了不同的目的而把页分为不同的类型，我们前面介绍的都是存储记录的数据页，其实还有很多其他类型的页，如表 5-5 所示。

表 5-5 其他类型的页

类型名称	十六进制	描述
FIL_PAGE_TYPE_ALLOCATED	0x0000	最新分配，还未使用
FIL_PAGE_UNDO_LOG	0x0002	undo 日志页
FIL_PAGE_INODE	0x0003	存储段的信息
FIL_PAGE_IBUF_FREE_LIST	0x0004	Change Buffer 空闲列表
FIL_PAGE_IBUF_BITMAP	0x0005	Change Buffer 的一些属性
FIL_PAGE_TYPE_SYS	0x0006	存储一些系统数据
FIL_PAGE_TYPE_TRX_SYS	0x0007	事务系统数据
FIL_PAGE_TYPE_FSP_HDR	0x0008	表空间头部信息
FIL_PAGE_TYPE_XDES	0x0009	存储区的一些属性
FIL_PAGE_TYPE_BLOB	0x000A	溢出页
FIL_PAGE_INDEX	0x45BF	索引页，也就是我们所说的数据页

用来存放记录的数据页的类型其实是 FIL_PAGE_INDEX，也就是索引页。至于啥是索引，且听下回分解。

小贴士

我们在前面唠叨记录的存储结构时，所说的溢出页的类型是 FIL_PAGE_TYPE_BLOB，而存放正常记录的页面类型是 FIL_PAGE_INDEX，两者是不一样的。

- FIL_PAGE_PREV 和 FIL_PAGE_NEXT：前文强调过，InnoDB 是以页为单位存放数据的，有时在存放某种类型的数据时，占用的空间非常大（比如一张表中可以有成千上万条记录）。InnoDB 可能无法一次性为这么多数据分配一个非常大的存储空间，而如果分散到多个不连续的页中进行存储，则需要把这些页关联起来，FIL_PAGE_PREV 和 FIL_PAGE_NEXT 就分别代表本数据页的上一个页和下一个页的页号。这样通过建立一个双向链表就把许许多多的页串联起来了，而无须这些页在物理上真正连着。需要注意的是，并不是所有类型的页都有上一个页和下一个页的属性，不过我们这里唠

叨的数据页（也就是类型为 FIL_PAGE_INDEX 的页）是有这两个属性的。所以，存储记录的数据页其实可以组成一个双向链表，如图 5-16 所示。

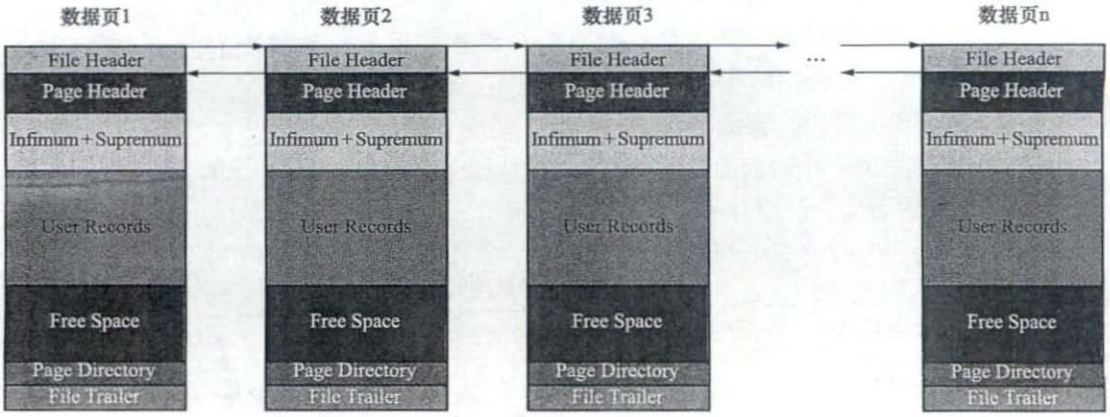

图 5-16 数据页组成的双向链表

关于 File Header 的其他属性暂时用不到，等后面用到的时候再提。

5.7 File Trailer（文件尾部）

我们知道，InnoDB 存储引擎会把数据存储到磁盘上，但是磁盘速度太慢，需要以页为单位把数据加载到内存中处理。如果该页中的数据在内存中被修改了，那么在修改后的某个时间还需要把数据刷新到磁盘中。但是，如果在刷新还没有结束的时候断电了该咋办，这不是相当尴尬么？为了检测一个页是否完整（也就是在刷新时有没有发生只刷新了一部分的尴尬情况），设计 InnoDB 的大叔在每个页的尾部都加了一个 File Trailer 部分，这个部分由 8 字节组成，可以分成 2 个小部分。

- 前 4 字节代表页的校验和。这个部分与 File Header 中的校验和相对应。每当一个页面在内存中发生修改时，在刷新之前就要把页面的校验和算出来。因为 File Header 在页面的前边，所以 File Header 中的校验和会被首先刷新到磁盘，当完全写完后，校验和也会被写到页的尾部。如果页面刷新成功，则页首和页尾的校验和应该是一致的。如果刷新了一部分后断电了，那么 File Header 中的校验和就代表着已经修改过的页，而 File Trailer 中的校验和代表着原先的页，二者不同则意味着刷新期间发生了错误。
- 后 4 字节代表页面被最后修改时对应的 LSN 的后 4 字节，正常情况下应该与 File Header 部分的 FIL_PAGE_LSN 的后 4 字节相同。这个部分也是用于校验页的完整性，不过我们目前还没说 LSN 是什么意思，所以大家可以先不用管这个属性。

这个 File Trailer 与 File Header 类似，都通用于所有类型的页。

5.8 总结

InnoDB 为了不同的目的而设计了不同类型的页，我们把用于存放记录的页称为数据页。

一个数据页可以被大致划分为 7 个部分，分别如下。

- File Header：表示页的一些通用信息，占固定的 38 字节。
- Page Header：表示数据页专有的一些信息，占固定的 56 字节。
- Infimum + Supremum：两个虚拟的伪记录，分别表示页中的最小记录和最大记录，占固定的 26 字节。
- User Records：真正存储我们插入的记录，大小不固定。
- Free Space：页中尚未使用的部分，大小不固定。
- Page Directory：页中某些记录的相对位置，也就是各个槽对应的记录在页面中的地址偏移量；大小不固定，插入的记录越多，这个部分占用的空间就越多。
- File Trailer：用于检验页是否完整，占固定的 8 字节。

每个记录的头信息中都有一个 next_record 属性，从而可以使页中的所有记录串联成一个单向链表。

InnoDB 会把页中的记录划分为若干个组，每个组的最后一个记录的地址偏移量作为一个槽，存放在 Page Directory 中，一个槽占用 2 字节。在一个页中根据主键查找记录是非常快的，分为两步。

1. 通过二分法确定该记录所在分组对应的槽，并找到该槽所在分组中主键值最小的那条记录。
2. 通过记录的 next_record 属性遍历该槽所在的组中的各个记录。

每个数据页的 File Header 部分都有上一个页和下一个页的编号，所以所有的数据页会组成一个双向链表。

在将页从内存刷新到磁盘时，为了保证页的完整性，页首和页尾都会存储页中数据的校验和，以及页面最后修改时对应的 LSN 值（页尾只会存储 LSN 值的后 4 字节）。如果页首和页尾的校验和以及 LSN 值校验不成功，就说明刷新期间出现了问题。